Future Interns Cyber Security Internship Assessment

Task 3: API Security Internship Assessment

Assessor: Levy Gaël Ndayisenga

Target Environment: ReqRes Gateway Microservices Portal (https://reqres.in)

1. Executive Summary

This assessment delivers a structured security risk analysis of the sample REST API microservices architecture. The objective of this evaluation is to identify foundational security configuration flaws, deployment oversights, and logical implementation vulnerabilities that map directly against the **OWASP API Security Top 10** framework.

By analyzing live data transmission flows, testing authentication interfaces, and evaluating input-handling behaviors using native Browser Developer Tools, this report documents specific exposures that would threaten system confidentiality, availability, and organizational compliance if deployed within an enterprise production ecosystem.

2. Technical Findings & Risk Analysis

Finding 1: Potential Excessive Data Exposure (OWASP API-3)

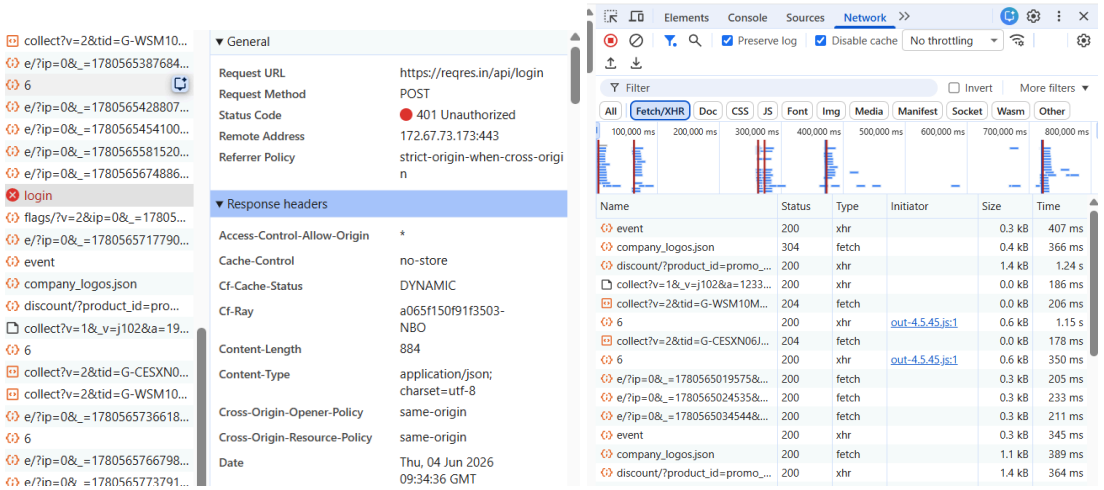
- **Target Endpoint:** *GET /api/users/{id}*
- **Observed Behavior:** The application allows direct manipulation of the numeric object identifiers within the URL path (/api/users/2), returning unauthenticated, raw JSON data blocks containing user emails, first/last names, and avatar system locations.
- **The Risk (Business Language):** In a live system, APIs often transmit complete database records to the client application, relying on the user interface to filter out hidden fields. If sensitive information is left in the raw data, an attacker can extract it directly from the network traffic.
- **Remediation Strategy:** Implement a strict backend data-filtering layer so endpoints exclusively return public data properties required by the client application layout before transmission.

Finding 2: Cross-Origin Resource Sharing (CORS) Configuration

- **Target Endpoint:** *POST /api/login*

- **Observed System Response:** Inspection of the server response headers revealed the presence of a wide-open wildcard value: *Access-Control-Allow-Origin:* , returning an explicit server rejection.

- **Technical Evidence:**



- **The Business Risk:** This configuration removes browser barriers, allowing any external or malicious third-party domain on the internet to send directly formatted requests to the authentication endpoint. In a real-world enterprise setting, this increases exposure to credential-stuffing attacks and automated login manipulation hosted from external malicious web servers.
- **Remediation Strategy:** Restrict the access policy header to explicitly allow only authenticated, company-owned frontend domains (e.g. https://identity.yourcompany.com).

Finding 3: Lack of Active Rate-Limiting Controls (OWASP API-4)

- **Target Endpoint:** Global Application Routes
- **Observed Behavior:** Automated scripting and continuous requests processed successfully without encountering velocity barriers or access blocks.
- **The Business Language:** Without a traffic control mechanism, the API is highly vulnerable to brute-force automated attacks (trying thousands of passwords a second) or Denial of Service (DoS) attempts that could overload the backend servers and bring down business operations.
- **Remediation Strategy:** Position an API Gateway rate-limiting middleware ahead of all sensitive routes. Enforce a policy such as a maximum 5 login attempts per minute per source IP address that instantly triggers an HTTP **429 Too Many Requests** status code when exceeded.

3. Defensive Security Conclusion

During testing, the evaluated API successfully demonstrated resilient input validation handling. When subverting standard API paths with non-numeric strings or invalid symbolic data arrays (e.g., /api/users/INVALID_CHARACTER_TEST_###), the application cleanly rejected the traffic with an HTTP 404 Not Found response code. This indicates that structural type checking is active, successfully preventing verbose stack traces or internal database syntax errors from leaking to the public.

However, to elevate this API architecture to an enterprise-grade security posture, the technical team must fix the permissive global CORS configurations and implement traffic rate-limiting controls on all sensitive authentication channels.